



Simply Rugby System

Portfolio of Evidence For Development

Contents

1) Introduction.....	3
2) Technical Achievements.....	3
3) Development Process.....	3
4) Good Working Practices.....	3
5) Ethical and Professional Considerations	4
6) Lessons Learned and Decisions Made	4

1) Introduction

The purpose of this document is to demonstrate the technical as well as professional skills used in this project. It includes all the concepts that were applied to ensure high-quality software. Each section corresponds to a different approach that contributed to creating the program in accordance with all good programming principles.

2) Technical Achievements

I believe that the entire application is a significant achievement for me. Many things didn't work at first, and I spent a considerable amount of time on several parts to gain a better understanding. The main technical achievements include my effort to follow good practices, such as using interfaces which didn't directly help in my current project but are valuable for the project's future development. I also implemented exception handling during file loading to ensure that an error is displayed when a file fails to load correctly. Using JFrame and JPanel was also a major challenge for me, as I hadn't had much practice with these technologies and components. Writing JUnit tests was also a challenge for me, as I wanted to try them out and see how they work. Additionally, I didn't know that it's not possible to perform proper unit testing on GUI components, which made the process even more difficult.

3) Development Process

To create this program, I used the following tools:

- Java (Swing)
- Eclipse IDE for Java Developers (includes Incubating components), Version: 2024-03 (4.31.0)
- JUnit 5
- I used Draw.io for all the diagrams included in this project.
- For version control, I used Git and GitHub, where I pushed all changes after finishing work to always have a backup just in case.
- I tried to follow the MVC pattern, which resulted in the project being split into packages, each with its own specific responsibility.

4) Good Working Practices

Before starting the project, I tried to deepen my understanding of the SOLID principles in Java to make this project as close as possible to a real work environment. I divided the project into two main packages: src and test, and structured the tests to reflect the classes they are testing by adding Test to the class names. I also wanted to create many more tests, but apart from login validation, additional tests did not make significant sense in this context. This aligns with best practices, where unit tests should be the most numerous, integration tests are much fewer, and end-to-end tests are used very rarely. The code is, in general, commented in a basic way. I'm not a fan of long comments, because if the code is well modularised and split into smaller classes etc., it's clear enough by itself.

5) Ethical and Professional Considerations

- i. The data used in the project was only for use by interested people, and most of it was made up.
- ii. The login is not very well secured because it has no connection with a real database, the only protection was validation of the username and password.
- iii. The application looks simple and very basic, but it meets all the requirements.
- iv. I tried to make the program as professional and intuitive for the user as possible.

6) Lessons Learned and Decisions Made

Every project gives a lot and teaches many different aspects. If I had much more time, which I didn't, I would like to build something bigger, but this gave me a good foundation for future projects that I already have in mind.

I know which areas I'm weak in, but I believe that is mostly because of insufficient experience, like building a detailed class diagram without writing any code. It's hard for me to imagine the whole application without seeing the code.

I decided to use a LinkedHashMap for storing skills, which keeps the values in order, that saved me a lot of extra coding.

At the beginning, even though I planned to create a coach session to store the data, I didn't do it, so later I suddenly had to modify the constructors to pass coach data as parameters.

I still have a lot to learn. The decisions I made are definitely not well-optimised, and my classes are still quite long, even though I wanted to avoid that. But with time, I'll improve at breaking things down into smaller parts.